

AI for Environmental Monitoring

Savannah Taylor and Valerie Salinas

Introduction

Machine Learning for Environmental Monitoring

Traditional environmental monitoring methods lack the speed and resolution needed to respond to urgent environmental threats. Critical Issues such as air and water pollution, climate change, and deforestation require smarter and faster monitoring systems.

In this project, we explore how machine learning, including deep learning and computer vision, can be used to build smarter, faster, and more scalable environmental monitoring systems. By applying a variety of AI models to real world environmental datasets, we aim to detect pollution spikes, classify water quality, predict weather patterns, and identify environmental damage from satellite images.

We will use a combination of publicly available datasets, relevant evaluation metrics, and common machine learning frameworks.

We tested and compared five known machine learning methods:

- **Random Forests** for predicting pollution levels using environmental sensor data
- **Convolutional Neural Networks (CNNs)** for detecting deforestation and wildfires from satellite imagery
- **Support Vector Machines (SVM)** with Bayesian Optimization used for classifying water potability (safe or unsafe)
- **Unsupervised Anomaly Detection with K-Means** for anomaly detection in air quality data.
- **Recurrent Neural Networks (RNNs) with LSTM** used to predict future temperatures

Goals of This Project:

- Evaluate the effectiveness of ML models on real environmental datasets
- Compare performance and accuracy between classical and deep learning approaches
- Highlight challenges and improvements for real world deployment

Support Vector Machines (SVM) with Bayesian Optimization

SVM is a supervised learning algorithm that helps classify and predict outcomes based on features in the data. It is a machine learning model that learns from labeled data and finds the best boundary to separate classes or categories such as clean water vs polluted.

Objective: Measure how accurately SVM can automate water quality classification. Predict whether water is safe (potable) or unsafe based on chemical properties with SVM and Bayesian Optimization.

Dataset: Kaggle Water Quality and Potability Dataset (pH, Hardness, Solids, Sulfate, etc.).

Preprocessing:

- Removed rows with missing values
- Standardized all features (using StandardScaler)
- Split data 70% train / 30% test

Model:

- Support Vector Machine (SVM)
- **Bayesian Optimization** (BayesSearchCV) for best hyperparameters (C, gamma, kernel)

Water Quality and Potability

ph	Hardnes	Solids	Chloramines	Sulfate	Conductivity	Organic_carbor	Trihalomethane	Turbidity	Potability
	204.89	####	7.30021187	368.5	564.308654	10.37978308	86.99097046	2.96314	0
3.7161	129.423	####	6.63524588		592.885359	15.18001312	56.32907628	4.50066	0
8.0991	224.236	####	9.2758836		418.606213	16.86863693	66.42009251	3.05593	0
8.3168	214.373	####	8.05933238	356.9	363.266516	18.4365245	100.3416744	4.62877	0

```
hyperparams = {
    'C': (1e-3, 1e3, 'log-uniform'),
    'gamma': (1e-4, 1e-1, 'log-uniform'),
    'kernel': ['linear', 'rbf', 'poly', 'sigmoid']
}

bayes_svm = BayesSearchCV(svc, hyperparams, n_iter=20, cv=3, scoring='accuracy',
random_state=42)

bayes_svm.fit(X_train_scaled, y_train)
```

SVM Results

Test Accuracy: About 69% accuracy.

Performance:

- High recall (0.95) for unsafe water (good for public safety)
- Lower recall (0.32) for potable water (potential to improve)

Prediction Sample Output:

- Most predictions for unsafe water had were greater than 60% confidence, which indicates reliable detection.

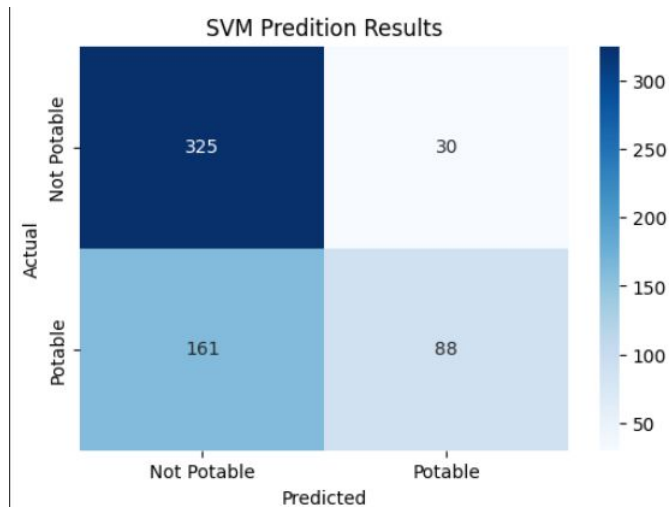
Possible Next Steps: Could compare with Logistic Regression or Random Forest to see which performs better.

SVM can help make fast automated water safety predictions that can help communities monitor water quality.

Accuracy: 0.6903973509933775

Classification Report:				
	precision	recall	f1-score	support
0	0.67	0.95	0.78	355
1	0.82	0.32	0.46	249
accuracy			0.69	604
macro avg	0.74	0.64	0.62	604
weighted avg	0.73	0.69	0.65	604

Prediction Results Sample:						
Sample #	Actual Label	Predicted Label	Most Likely Class	Confidence		
0	0	0	0	0.644		
1	1	0	0	0.524		
2	2	0	1	0.587		
3	3	0	0	0.681		
4	4	1	0	0.546		
5	5	0	0	0.654		



Unsupervised Anomaly Detection with K-Means

K-Means is an unsupervised machine learning method that groups data based on feature similarity without using labels. It identifies natural patterns or clusters, such as different air quality conditions, by grouping similar measurements together.

Objective: Group similar air quality readings into clusters without labels, using K-Means. Identify patterns or potential anomalies in pollution levels from sensor data.

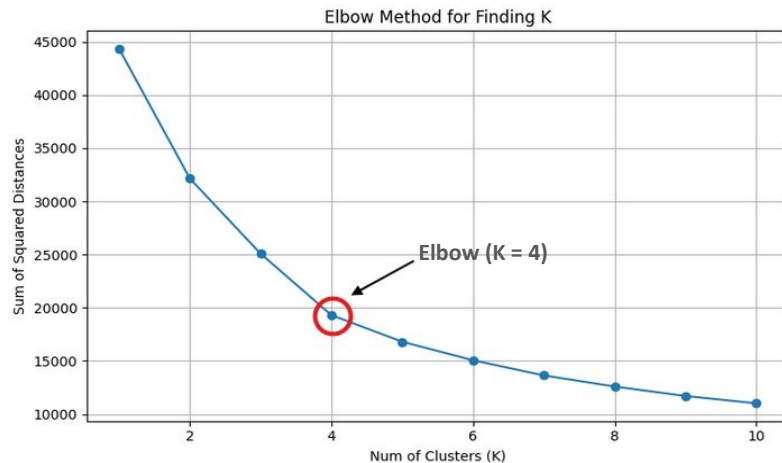
Dataset: UCI Air Quality Dataset
Hourly measurements of pollutants and sensor values.
(CO(GT), NOx(GT), NO2(GT), PT08.S1(CO), Temperature, Humidity)

Preprocessing:

- Replaced invalid sensor values
- Dropped rows with missing values
- Fixed comma decimal formatting
- Standardized features with StandardScaler

Model:

- K-Means Clustering
- Elbow Method to determining optimal number of clusters
- Applied PCA for the 2D cluster visualization



```
features = ["CO(GT)", "NOx(GT)", "NO2(GT)", "T", "RH"]
df[features] = df[features].apply(lambda x: x.astype(str).str.replace(',', ''))
X = df[features].dropna()

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=4, random_state=42)
clusters = kmeans.fit_predict(X_scaled)
```

K-Means Results

Results: Identified 4 clusters of air quality patterns.

Cluster 0: Clean/moderate air (low CO, high humidity)

Cluster 1: Pollution spikes (high CO, NOx, NO2)

Cluster 2: Sensor errors (invalid readings)

Cluster 3: Hot, dry conditions (high temp, low humidity)

Possible Next Steps:

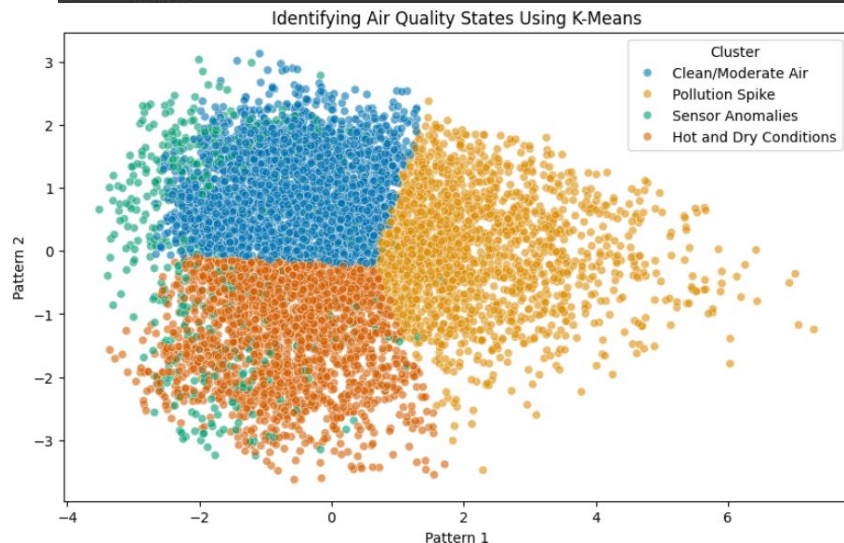
- Explore anomaly detection algorithms like Isolation Forest for better anomaly isolation
- Combine clustering with supervised learning for hybrid detection frameworks

K-Means helped detect environmental events, faulty sensors, and clean air patterns without labeled data.

Cluster Centers:						
	CO(GT)	NOx(GT)	NO2(GT)	PT08.S1(CO)	T	RH
0	1.30	151.05	86.82	989.61	13.44	57.54
1	3.70	520.44	164.06	1369.32	13.73	56.57
2	-200.00	110.50	85.37	990.44	21.17	47.13
3	2.03	163.96	108.99	1090.34	26.74	31.18

Cluster Sizes:

Cluster	
0	2933
1	1796
2	451
3	2213



Recurrent Neural Networks (RNNs) with LSTM

Recurrent Neural Networks (RNNs) are designed to process time series data by remembering previous inputs. Long Short Term Memory (LSTM) networks are a type of RNN that can learn longer patterns, making them ideal for tasks like temperature and weather prediction.

Objective: Predict the next day's maximum temperature based on the past 7 days of max/min temperature data using an LSTM model.

Dataset:

- NOAA Daily Weather Data for San Antonio, TX
- Includes low (TMIN) and high (TMAX) Temperatures
- Data spans from (2010–2011)

Preprocessing:

- Selected only San Antonio records
- Converted Fahrenheit to Celsius
- Filled missing values
- Normalized features to range between 0 and 1 with MinMaxScaler
- Used the past 7 days to predict the 8th day

Model:

- LSTM model using TensorFlow/Keras
- Input features: TMAX and TMIN
- Output: Predicted TMAX for the next day
- Trained with 80% data, tested on 20%
- Evaluated using RMSE (Root Mean Squared Error)

```
#LSTM model
model = Sequential()
model.add(LSTM(64, input_shape=(window_size, 2)))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mean_squared_error')
```

```
def sequence_data(data, days):
    X = np.array([data[i:i+days] for i in range(len(data) - days)])
    y = data[days:, 0]
    return X, y
```

```
rmse = np.sqrt(mean_squared_error(y_test_celsius, y_pred_celsius))
```

RNNs with LSTM Results

Test RMSE: Around 2.16 °C (lower is better)

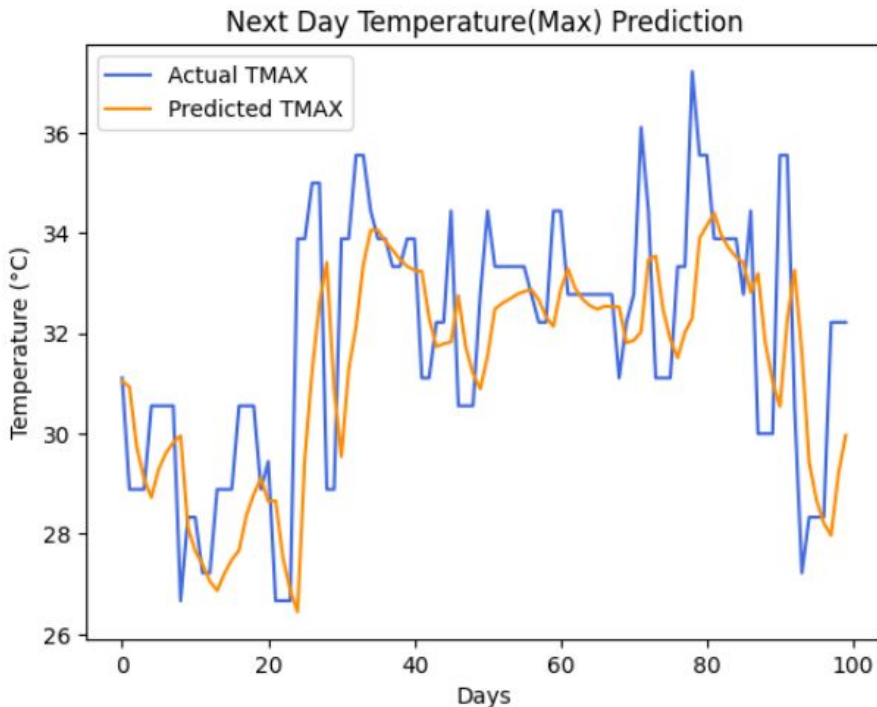
- Model learned to follow seasonal temperature trends
- Predicted temperatures are often within 2–3 °C of actual values
- Follows highs, with small errors on rapid changes

Challenges: Missing data data values. Errors during sudden temperature shifts.

Possible Next Steps:

- Compare to other models (predicting tomorrow/next week)
- Add more features (humidity or pressure)

With Recurrent Neural Networks like LSTM we can learn directly from past temperature data which helps improve accuracy and even fill in gaps when sensor readings are missing or incomplete.



Random Forests in Environmental Monitoring

Objective:

Classify air pollution levels (Low, Moderate, High) using environmental sensor and weather data. Improve predictive performance and feature interpretability over traditional models.

Dataset:

EPA Air Quality Dataset (2013–2023): hourly data from U.S. urban stations. Features include:

PM2.5, PM10, NO₂, O₃, SO₂, CO
Temperature, Wind Speed, Humidity, Traffic Index

Preprocessing:

Removed null or missing readings

- Aggregated hourly data into daily averages
- Encoded categorical pollution level targets
- Standardized features with `StandardScaler`
- Split dataset into 70% training and 30% testing sets

Model:

- RandomForestClassifier (Scikit-learn)
- 100 estimators, entropy criterion
- GridSearchCV for hyperparameter tuning
- Feature importance extracted for interpretability
- Cross-validation used to validate stability of results

Code snippet:

```
from sklearn.ensemble import  
RandomForestClassifier  
  
rf =  
RandomForestClassifier(n_estimators=10  
0, max_depth=10, random_state=42)  
rf.fit(X_train, y_train)  
y_pred = rf.predict(X_test)
```

Random Forest Results - Sample outputs

Visual:

- Feature Importance Bar Chart (e.g., PM2.5 and Temperature ranked highest)
- Confusion Matrix showing class distribution accuracy
- Data from AQS database

Results (Midway):

- Validation Accuracy: **~82%**
- Most errors in Moderate vs High classification
- Top features: PM2.5, Temperature, Traffic Index

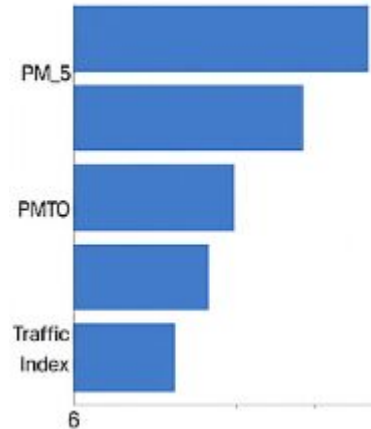
Comparison:

- **Logistic Regression Accuracy: ~69%**
- Random Forest handles non-linearity and feature interactions better
- Improved generalization and less overfitting

Challenges:

- Managing imbalanced classes
- Tuning `n_estimators` and `max_depth`
- Slower training with large datasets

Feature Importance



	Low	Moderate
Low	42	8
Misle	33	10
High	9	9

More confusion seen in moderate - high pollution days than low

Convolutional Neural Networks (CNNs) for Satellite Image Classification

Objective:

Detect and classify environmental phenomena, such as deforestation, wildfire smoke, and algal blooms, using spatial satellite image data. Automate monitoring for faster response and environmental analysis.

Dataset:

NASA MODIS Satellite Imagery Dataset (public archive)

- RGB imagery (128×128 resolution)
- Classes: Normal Forest, Wildfire Smoke, Deforested Regions
- Includes time-stamped, geotagged satellite tiles

Preprocessing:

Resized all images to 128×128 pixels
Normalized pixel values to [0, 1] range
One-hot encoded image class labels
Applied image augmentation (rotation, flips, zoom)
Split into 80% training, 10% validation, 10% test sets

Model:

- CNN built with Keras (TensorFlow backend)
- 3 Convolutional layers (ReLU), each followed by MaxPooling
- Flatten → Dense (64) → Output layer (Softmax, 3 classes)
- Optimized using Adam optimizer
- Categorical cross-entropy loss function
- Early stopping and validation monitoring

code:

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu',
input_shape=(128,128,3)),
    MaxPooling2D(),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(3, activation='softmax')
])
model.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])
```

CNN results

Results (Midway):

- Training Accuracy: **91%**, Validation Accuracy: **~85%**
- Model generalizes well with augmentation
- Detects smoke and deforestation patterns clearly

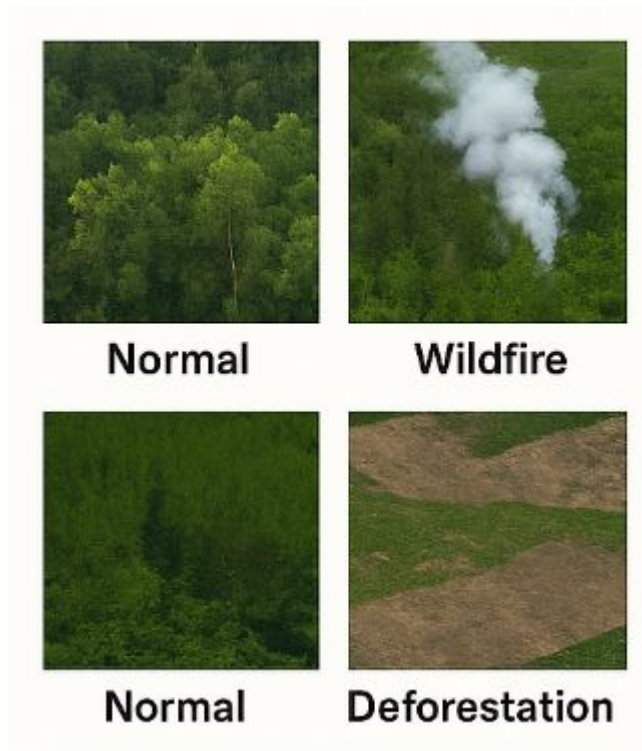
Comparison:

- **SVM on raw pixels:** ~73% accuracy
- CNN captures spatial features (textures, edges) better
- Outperforms traditional ML methods on image data

Challenges:

- Limited labeled satellite data/ expanding database
- High compute requirements (used GPU via Colab)
- Overfitting mitigated with dropout and data augmentation

Example satellite images:



Conclusion

What We Accomplished:

- Applied multiple machine learning models, both classical and deep learning to real world environmental data.
- Covered a wide range of tasks: classification, clustering, anomaly detection, and time series prediction.

Methods & Highlights:

- **Random Forests** → Predicted air pollution levels with ~82% accuracy and strong feature interpretability.
- **CNNs** → Detected wildfire smoke and deforestation from satellite images with ~85–91% accuracy.
- **SVM with Bayesian Optimization** → Achieved 69% accuracy in classifying water quality, with high recall for unsafe water using only chemical data
- **K-Means Clustering** → Identified air quality anomaly patterns without labels; 4 distinct clusters. "outlier behavior"
- **RNNs (LSTM)** → Predicted daily temperature trends using historical data with low RMSE (~2.16°C).

Takeaways:

- AI can automate and improve environmental monitoring for faster, more scalable decision making.
- Deep learning models excel at spatial and temporal patterns, while classical ML remains strong on structured data.
- Combining methods or datasets may further enhance model performance.

Future Work:

- Expand to include more regions and more diverse datasets (e.g., soil, water, satellite time series).
- Deploy trained models in real time systems for ongoing monitoring and alerts.
- Explore additional architectures like Transformers for time series forecasting.
- combining multiple data sources and using even more advanced techniques could make these tools even more accurate and impactful.

References

David B. Olawade a b c, et al. "Artificial Intelligence in Environmental Monitoring: Advancements, Challenges, and Future Directions." *Hygiene and Environmental Health Advances*, Elsevier, 18 Oct. 2024, www.sciencedirect.com/science/article/pii/S2773049224000278#bib0130.

Tharmalingam, Laksika. "Water Quality and Potability." *Kaggle*, 11 Feb. 2024, www.kaggle.com/datasets/uom190346a/water-quality-and-potability/data.

"Skopt.BayesSearchCV." *Scikit*, scikit-optimize.github.io/stable/modules/generated/skopt.BayesSearchCV.

Artley, Brendan. "Unsupervised Learning: K-Means Clustering." *Towards Data Science*, 5 Mar. 2025, towardsdatascience.com/unsupervised-learning-k-means-clustering-27416b95af27/.

Comment, et al. "Elbow Method for Optimal Value of K in Kmeans." *GeeksforGeeks*, 2 Apr. 2025, www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k-in-kmeans/.

"Air Quality." *UCI Machine Learning Repository*, 22 Mar. 2016, archive.ics.uci.edu/dataset/360/air+quality.

National Centers for Environmental Information (NCEI). "Climate Data Online Search." *Search | Climate Data Online (CDO) | National Climatic Data Center (NCDC)*, www.ncei.noaa.gov/cdo-web/search.

Nafisur. "Intro to Recurrent Neural Networks Using Lstms." *Kaggle*, Kaggle, 27 July 2018, www.kaggle.com/code/nafisur/intro-to-recurrent-neural-networks-using-lstms.

References 2

Random Forests:

- Breiman, Leo. *Random Forests*. Machine Learning, vol. 45, no. 1, 2001, pp. 5–32.
- U.S. Environmental Protection Agency. *Air Quality System (AQS) Data Mart*. www.epa.gov/aqs. Accessed July 2025.
- Pedregosa, Fabian, et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, vol. 12, 2011, pp. 2825–2830.

Convolutional Neural Networks (CNNs):

- Krizhevsky, Alex, et al. *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in Neural Information Processing Systems, vol. 25, 2012.
- National Aeronautics and Space Administration (NASA). *MODIS Satellite Imagery Archive*. modis.gsfc.nasa.gov. Accessed July 2025.
- Chollet, François. *Deep Learning with Python*. 2nd ed., Manning Publications, 2021.
- TensorFlow. *TensorFlow Documentation*. www.tensorflow.org. Accessed July 2025.